

# Programming Assignment

Automata Theory Monsoon 2026, IIIT Hyderabad

August 8, 2026

**Total Points:** 150 Points

**Deadline:** 9 September 2026

---

**General Instructions:** Read the input and output formats very carefully since the evaluation will be **scripted**. The goal is not to stress test your code but to see if you have understood the underlying concepts used in the implementation. There will be an evaluation/viva after the submission, and a part of the grade would depend on it.

**Language constraints:** The submission must be in Python. For any further use of other libraries please refer to one of the TAs for approval. Usage of any other language will lead to no evaluation.

---

## 1 RegExpert

[75 Points]

We've all used programs like **grep** before *[iss :']*. But have you ever wondered how they actually work? Well, you're about to find out (*unless* you end up using a LLM).

In this assignment you will design and implement a program that accepts a **regular expression** and a **block of text** as input, and outputs all the words in the text that match the expression.

### How a regex engine actually works

A typical regex engine processes input in three stages:

**Stage 1: Parse** the regular expression. This can be as simple as converting the expression into postfix notation using the shunting-yard algorithm, or if you want to be fancy building an explicit abstract syntax tree. Either representation is acceptable as long as the next stage can consume it unambiguously.

**Stage 2: Regex  $\rightarrow$  NFA**, using **Thompson's construction**.

**Stage 3: NFA  $\rightarrow$  DFA**, using the **subset construction** taught in class. You then simulate the input text against this DFA to determine, word by word, whether it is accepted.

### Thompson's construction

Thompson's construction builds an NFA fragment for each operator by combining the NFA fragments of its operands using  $\epsilon$ -transitions. Each fragment has exactly one start state and one accept state, which makes the fragments easy to compose. The four base cases are shown below.

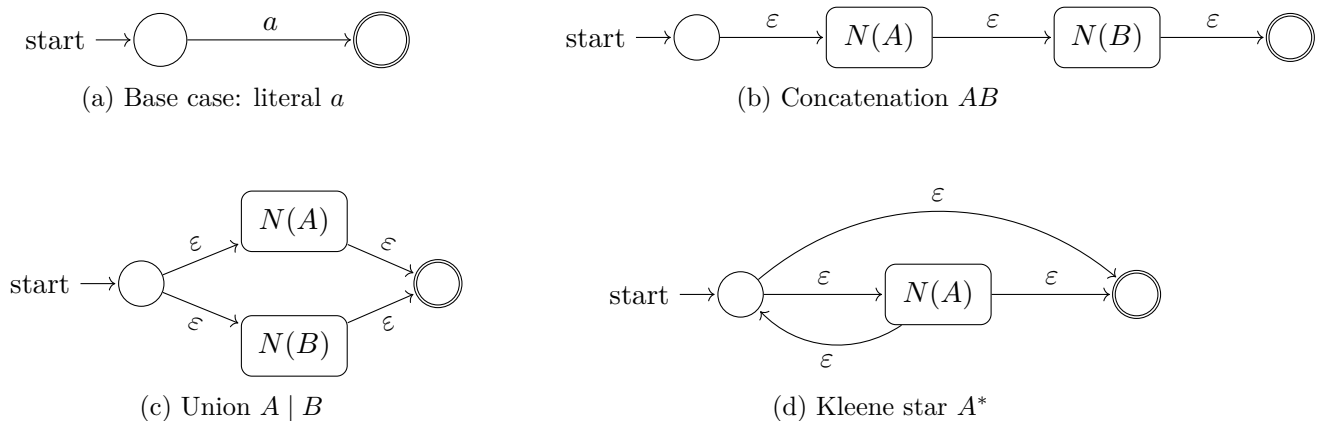


Figure 1: Thompson's construction: the four base fragments.  $N(A)$  denotes the already-built NFA fragment for sub-expression  $A$ , drawn as a black box with a single entry and single exit.

### Subset construction

Given an NFA with  $\epsilon$ -transitions, the subset construction builds an equivalent DFA whose states are *sets of NFA states*.

**$\epsilon$ -closure:** Before building the DFA transitions, you must compute the  $\epsilon$ -closure of a state. For any NFA state  $q$ , the  $\epsilon$ -closure( $\{q\}$ ) is the set of all NFA states reachable from  $q$  by following zero or more  $\epsilon$ -transitions. This represents all the states the NFA could simultaneously occupy without consuming any new characters from the input text.

The construction rules are:

- The DFA start state is  $\epsilon$ -closure( $\{q_0\}$ ), where  $q_0$  is the NFA start state.
- For a DFA state  $S$  and input symbol  $c$ , the transition is  $\delta_{\text{DFA}}(S, c) = \epsilon$ -closure( $\bigcup_{q \in S} \delta_{\text{NFA}}(q, c)$ ).
- A DFA state is accepting iff it contains at least one NFA accept state.
- New DFA states are discovered lazily by exploring reachable sets; the process terminates because there are finitely many subsets of a finite NFA state set.

### Supported constructs

To keep the assignment squarely inside regular languages, your engine must support exactly this grammar over an alphabet  $\Sigma$  consisting of all printable ASCII characters, excluding whitespace and the specific metacharacters listed below:

| Construct  | Meaning                 |
|------------|-------------------------|
| $a$        | literal character       |
| $AB$       | concatenation           |
| $A \mid B$ | union                   |
| $A^*$      | Kleene star (0 or more) |

All of the following must *also* be implemented, but they are purely **syntactic sugar**: they exist to make the regex-writer's life easier, and each one can be mechanically decomposed into the four core operations above before (or during) NFA construction. For example, the operation  $A^+$  (one or more occurrences) decomposes into  $AA^*$ . Similarly, the grouping operator  $(A)$  simply controls evaluation precedence during the parsing stage.

| Construct                                     | Meaning          |
|-----------------------------------------------|------------------|
| $A^+$                                         | one or more      |
| $A^?$                                         | zero or one      |
| $(A)$                                         | grouping         |
| $[abc]$                                       | character class  |
| $[a-z]$                                       | character range  |
| $\backslash . \backslash * \backslash ($ etc. | escaped literals |

## Matching Rules

- The program scans the input text and tokenizes it into **words**, where a word is a maximal run of non-whitespace characters. Punctuation stays attached to the word it borders (no separate punctuation handling is required).
- A word is reported as a **match** if and only if the *entire* word is accepted by the final DFA, i.e., running the whole word through the DFA, character by character, ends in an accepting state. Partial / substring matches inside a word do not count.
- Matching is **case-sensitive** unless the regular expression itself uses a character class that covers both cases (e.g.  $[a-zA-Z]$ ).
- If the regular expression is malformed, the program must print a clear parse error and exit gracefully, it must not crash or hang. Malformed expressions include missing operands (e.g.,  $A |$ ), stacked quantifiers (e.g.,  $A^{**}$ ), and unsupported operators (e.g.,  $[\^A]$ ).

## Input and Output Format

**Input:** Read from standard input (`stdin`). The first line of the input will be a single string representing the regular expression. All subsequent lines will contain the text to be searched. The text block has no explicit terminator or line count. Read from standard input until end-of-file (EOF) to capture it in full.

**Output:** Print each matching word to standard output (`stdout`), one word per line, in the exact order they appear in the text.

## Constraints

**Strictly No Backtracking:** You are strictly prohibited from implementing a recursive backtracking (Depth-First Search) regex solver. You must programmatically construct the NFA and DFA data structures as taught in class. Furthermore, using any built-in regex libraries (such as `re` in Python) will result in an immediate zero.

## Debug Mode and Output Format

To allow the autograder to verify that you have successfully constructed the automata, your program must accept a `--debug` command-line flag.

When this flag is present, your program must print your final **DFA Transition Matrix** to standard error (`stderr`) *before* it evaluates the text and prints matches to `stdout`.

The transition matrix must be printed in a strict Comma-Separated Values (CSV) format:

- **Header row:** The word **State**, followed by every character in the alphabet  $\Sigma$  that is actively used in the regular expression, sorted in ascending ASCII order.
- **State rows:** The integer ID of the DFA state, followed by the destination state ID for each character in the alphabet.
- **State labels:** Prefix the start state with `->` and accept states with an asterisk `*`. (If the start state is also an accept state, use `->*`).
- **Dead transitions:** Print a dash (`-`) if a transition leads to a dead/reject state. Do not include the dead state as a row in your matrix.

### Example Debug Output for the regex `ab*`:

```
State, a, b
->0, 1, -
*1, -, 2
*2, -, 2
```

**Note on State Numbering:** Do not worry if your specific integer state IDs do not match the example exactly. Because different implementations may number NFA states differently or discover DFA subsets in a different order, your state numbers might be permuted.

The autograder will evaluate your matrix based on its *topological structure* and equivalence, not the exact integer labels. However, you **must** sort the alphabet columns in ASCII order as specified above.

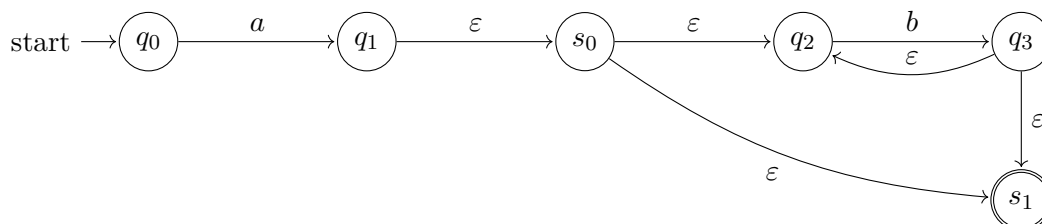
## Worked example

### Input:

```
ab*
a ab abb abbb c
```

### Step 1: NFA via Thompson's construction

Building  $N(a)$ ,  $N(b)$ , then  $N(b^*)$ , then concatenating  $N(a)$  with  $N(b^*)$  gives an NFA with states  $\{q_0, q_1, s_0, q_2, q_3, s_1\}$ :



## Step 2: DFA via subset construction

| DFA state      | NFA subset               | on <i>a</i> | on <i>b</i> |
|----------------|--------------------------|-------------|-------------|
| $D_0$ (start)  | $\{q_0\}$                | $D_1$       | dead        |
| $D_1$ (accept) | $\{q_1, s_0, q_2, s_1\}$ | dead        | $D_2$       |
| $D_2$ (accept) | $\{q_3, q_2, s_1\}$      | dead        | $D_2$       |
| dead           | $\emptyset$              | dead        | dead        |

## Step 3: tokenize and simulate

Tokenizing the text gives the words: a, ab, abb, abbb, c.

| Word | DFA run                                                                               | Result |
|------|---------------------------------------------------------------------------------------|--------|
| a    | $D_0 \xrightarrow{a} D_1$                                                             | accept |
| ab   | $D_0 \xrightarrow{a} D_1 \xrightarrow{b} D_2$                                         | accept |
| abb  | $D_0 \xrightarrow{a} D_1 \xrightarrow{b} D_2 \xrightarrow{b} D_2$                     | accept |
| abbb | $D_0 \xrightarrow{a} D_1 \xrightarrow{b} D_2 \xrightarrow{b} D_2 \xrightarrow{b} D_2$ | accept |
| c    | $D_0 \xrightarrow{c} \text{dead}$                                                     | reject |

## Expected output

```
a
ab
abb
abbb
```

## Additional test cases

### Test Case 1

#### Input:

```
[a-z]+ing
I was running while thinking about swimming laps.
```

#### Expected Output:

```
running
thinking
swimming
```

### Test Case 2

#### Input:

```
(a|b)*abb
aabb babb ab abb
```

#### Expected Output:

```
aabb  
babb  
abb
```

### Test Case 3

#### Input:

```
[0-9]+(\\. [0-9]+)?  
Price is 42 or 3.14 or item42
```

#### Expected Output:

```
42  
3.14
```

### Test Case 4

#### Input:

```
colou?r  
I prefer color over colour, but colour is a typo.
```

#### Expected Output:

```
color
```

**Note:** *colour is not printed in the output because we consider the word including the punctuation. So the word would be 'colour,'.*

### Test Case 5

#### Input:

```
[  
anything
```

**Expected Output:** *No output to stdout. Program must print a parse error to stderr and exit with a non-zero code.*

## 2 Syntactic Analysis

[75 Points]

Link to the boilerplate

You are tasked with building a compiler for a simple robot-control programming language. The language supports the following token types:

- Identifiers
- Keywords
- Integers
- Floating-point numbers
- Symbols

**NOTE:** You have been provided boilerplate code that already implements the lexical analysis (tokenization) step. Your task is strictly to implement the `checkGrammar(tokens)` function, which takes the list of generated tokens and performs syntactic analysis.

- **Tokenization (Provided):** The provided boilerplate uses Finite State Automata (FSAs) to read the source code and classify tokens as `IDENTIFIER`, `KEYWORD`, `INTEGER`, `FLOAT`, or `SYMBOL`. It also enforces token hierarchy (e.g., distinguishing keywords from identifiers) and raises lexical errors for invalid inputs.
- **Syntactic Analysis (Your Task):** You must write a recursive-descent parser within `checkGrammar(tokens)` that validates the token sequence against the context-free grammar (CFG) provided below.

### Rules for Syntactic Analysis

The language grammar is defined by the following context-free grammar (CFG):

$$\begin{aligned} S &\rightarrow P \\ P &\rightarrow \text{statement } P \mid \epsilon \\ \text{statement} &\rightarrow \text{Move} \mid \text{Turn} \mid \text{Set} \mid \text{Print} \mid \text{If} \\ \text{Move} &\rightarrow \text{move } x ; \\ \text{Turn} &\rightarrow \text{turn left} ; \mid \text{turn right} ; \\ \text{Set} &\rightarrow \text{set } id = x ; \\ \text{Print} &\rightarrow \text{print } x ; \\ \text{If} &\rightarrow \text{if } x \text{ RelOp } x \text{ Move ElsePart} \\ \text{ElsePart} &\rightarrow \text{else Move} \mid \epsilon \\ \text{RelOp} &\rightarrow < \mid > \mid == \\ x &\rightarrow \text{integer} \mid \text{float} \mid id \end{aligned}$$

**Keywords:** move, turn, left, right, set, print, if, else.

## I/O Format

### Input

Your program should accept a single line of source code as input.

### Output

If the source code is both lexically and syntactically valid, print every recognized token in the following format, one token per line. The token type must be one of: IDENTIFIER, KEYWORD, INTEGER, FLOAT, or SYMBOL.

```
Token Type: KEYWORD, Token Value: move
Token Type: INTEGER, Token Value: 10
Token Type: SYMBOL, Token Value: ;
```

### Error Handling Requirements

- **Lexical Error:** If tokenization fails, raise a `ValueError` with a brief and descriptive message. Do not print partially recognized tokens.
- **Syntactic Error:** If tokenization succeeds but the sequence violates the CFG grammar, print all successfully recognized tokens first, then raise a `SyntaxError` with a brief description.

### Test Cases

#### Test Case 1

##### Input:

```
move 10; turn left;
```

##### Expected Output:

```
Token Type: KEYWORD, Token Value: move
Token Type: INTEGER, Token Value: 10
Token Type: SYMBOL, Token Value: ;
Token Type: KEYWORD, Token Value: turn
Token Type: KEYWORD, Token Value: left
Token Type: SYMBOL, Token Value: ;
```

#### Test Case 2

##### Input:

```
if x == 10 move 20; else move 30;
```

##### Expected Output:

```
Token Type: KEYWORD, Token Value: if
Token Type: IDENTIFIER, Token Value: x
Token Type: SYMBOL, Token Value: ==
Token Type: INTEGER, Token Value: 10
Token Type: KEYWORD, Token Value: move
Token Type: INTEGER, Token Value: 20
```

```
Token Type: SYMBOL, Token Value: ;  
Token Type: KEYWORD, Token Value: else  
Token Type: KEYWORD, Token Value: move  
Token Type: INTEGER, Token Value: 30  
Token Type: SYMBOL, Token Value: ;
```

### Test Case 3

#### Input:

```
set 2abc = 5;
```

#### Expected Output:

```
ValueError: Invalid identifier: 2abc  
Identifier can't start with digits
```

### Test Case 4

#### Input:

```
set x = ;
```

#### Expected Output:

```
Token Type: KEYWORD, Token Value: set  
Token Type: IDENTIFIER, Token Value: x  
Token Type: SYMBOL, Token Value: =  
Token Type: SYMBOL, Token Value: ;  
SyntaxError: Expected integer, float, or identifier after '='
```

### Test Case 5

#### Input:

```
else move 10;
```

#### Expected Output:

```
Token Type: KEYWORD, Token Value: else  
Token Type: KEYWORD, Token Value: move  
Token Type: INTEGER, Token Value: 10  
Token Type: SYMBOL, Token Value: ;  
SyntaxError: 'else' occurs before 'if'
```

## Grading and Submission Details

### Grading Scheme

1. Question 1 (75 points)
  - (a) RegExpert (50 points)
  - (b) Viva (25 points)
2. Question 2 (75 points)
  - (a) Syntactic Analysis (50 points)
  - (b) Viva (25 points)

### Submission details

A new portal will be created for the Programming Assignment. All your files should be placed in a single zip file named as "<Roll\_Number>.zip". The folder "<Roll\_Number>" **must** follow this format for the automated grader to work.

```
<Roll_Number>/
|
|-- Question-1/
|   |-- regexpert.py           # RegExpert implementation
|
|-- Question-2/
|   |-- q2.py                 # Implementation for Question 2
|
|-- README.md                 # Brief explanation of solutions
```